

27. for : la boucle qui compte

Départ, condition, pas : les trois morceaux d'une boucle qui compte, sur une seule ligne.

1. Ce qu'il faut comprendre

Compter est si fréquent que le C++ lui donne sa boucle : `for (uint8_t i = 0; i < 20; i++)`. Trois morceaux, séparés par des **points-virgules** :

1. le départ `uint8_t i = 0` — exécuté une fois, avant tout. **2. la condition** `i < 20` — posée avant chaque tour, comme dans `while`. **3. le pas** `i++` — exécuté à la fin de chaque tour. Le `for` est **exactement** ce `while` -ci, rangé sur une ligne :

`uint8_t i = 0; while (i < 20) { ...; i++; }` — tout ce qui concerne le comptage est au même endroit, et on ne risque plus d'oublier le `i++` en bas du bloc.

`i < 20` fait **20 tours**, de 0 à 19 — pas de 1 à 20. L'écran a 20 colonnes, numérotées de 0 à 19 : la boucle et l'écran parlent la même langue. `i` n'existe **que dans la boucle** : après l'accolade, le nom est libre.

Le pas n'est pas forcément `i++` : `i += 2` saute une case sur deux. Et `i` sert dans des calculs : `nombre(i * 2, ...)` écarte les chiffres.

Une question : la boucle n'appelle jamais `image()`, et pourtant elle ne va pas aussi vite que le processeur le pourrait. C'est **l'écriture à l'écran** qui attend, pas la boucle : chaque `texte()` attend la courte pause entre deux images. La leçon suivante explique pourquoi — c'est **la** règle de la Game Boy.

À toi : trace une colonne de # (la ligne varie, la colonne est fixe) de la ligne 12 à la ligne 17.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **518 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	— la mise en route de la console
main()	120 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre	— attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), nombre()	— trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre()	— écrit un nombre en chiffres
CopierTuiles	40 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran

```

Tuiles          176 o ← CopierTuiles  - 11 lettres sur 43
"#"             1 o ← texte()  - un texte gravé
"#"             1 o ← texte()  - un texte gravé
Lettres gravées : 0123456789#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,
Notes
Fonctions du langage appelées :
texte() × 2      36 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
nombre() × 1     22 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : dessine deux rangées de # et une rangée de chiffres, avec « for ».
//
// Ce qui est nouveau : la boucle for, la boucle qui COMPTE.
//   for (DÉPART ; CONDITION ; PAS) ...
//   DÉPART   : fait une fois, au début      (uint8_t i = 0 : i commence à 0)
//   CONDITION : testée avant chaque tour    (i < 20 : on continue tant que i < 20)
//   PAS      : fait à la fin de chaque tour (i++ : i gagne 1)

#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {                                // Le jeu commence ici.
    // Vingt tours : i vaut 0, 1, 2 ... 19.
    for (uint8_t i = 0; i < 20; i++) {
        texte(i, 3, "#");                  // Un # en colonne i, ligne 3 : toute la ligne.
    }                                       // Quand i vaut 20, « 20 < 20 » est faux : fini.

    // Un pas de deux : 0, 2, 4 ... 18.
    for (uint8_t i = 0; i < 20; i += 2) {   // « i += 2 » = « i = i + 2 ».
        texte(i, 5, "#");                  // Un # une colonne sur deux.
    }

    // i sert aussi dans le calcul.
    for (uint8_t i = 0; i < 10; i++) {      // i vaut 0, 1, 2 ... 9.
        nombre(i * 2, 8, i, 1);            // En colonne i * 2 : 0, 2, 4 ... 18,
                                           // on écrit i sur 1 chiffre : 0 1 2 3 ... 9.
    }

    while (true) {                          // La boucle du jeu : l'écran reste allumé.
        image();
    }
}

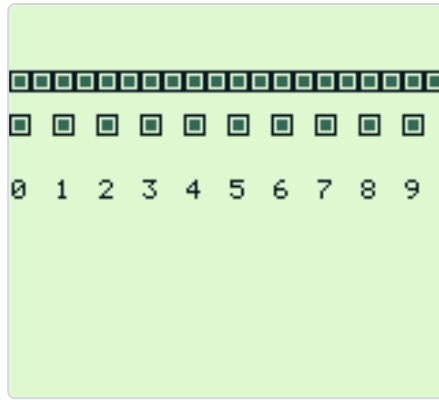
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une ligne pleine de #, une ligne d'un # sur deux, et les chiffres de 0 à 9 espacés — qui se dessinent sous tes yeux.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ 20 tours : la ligne est pleine
- ✓ un pas de 2 : une case sur deux
- ✓ i vaut 0 au premier tour, 9 au dernier

Ensuite : **27.1. La fonction `ecran()` — éteindre et rallumer l'écran** — chapitre 5 : Les boucles : répéter. · `gameboy3` — écrire une cartouche Game Boy en C++